

# tRPC-Agent

## 智能体应用开发框架

2026年5月



# 目录

- tRPC-Agent 介绍
- tRPC-Agent 上手 Issue
- tRPC-Agent 实战任务
- tRPC-Agent 代码提交流程
- 总结

# 1. tRPC-Agent 简介

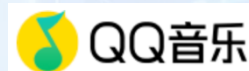
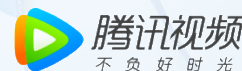
## 生产级 Agent 应用开发框架

### 支持两种语言

- Python: <https://github.com/trpc-group/trpc-agent-python>
- Go: <https://github.com/trpc-group/trpc-agent-go>

把 Agent 从 Demo 做成可编排、可治理、可评测、可服务化的工程系统。

用户

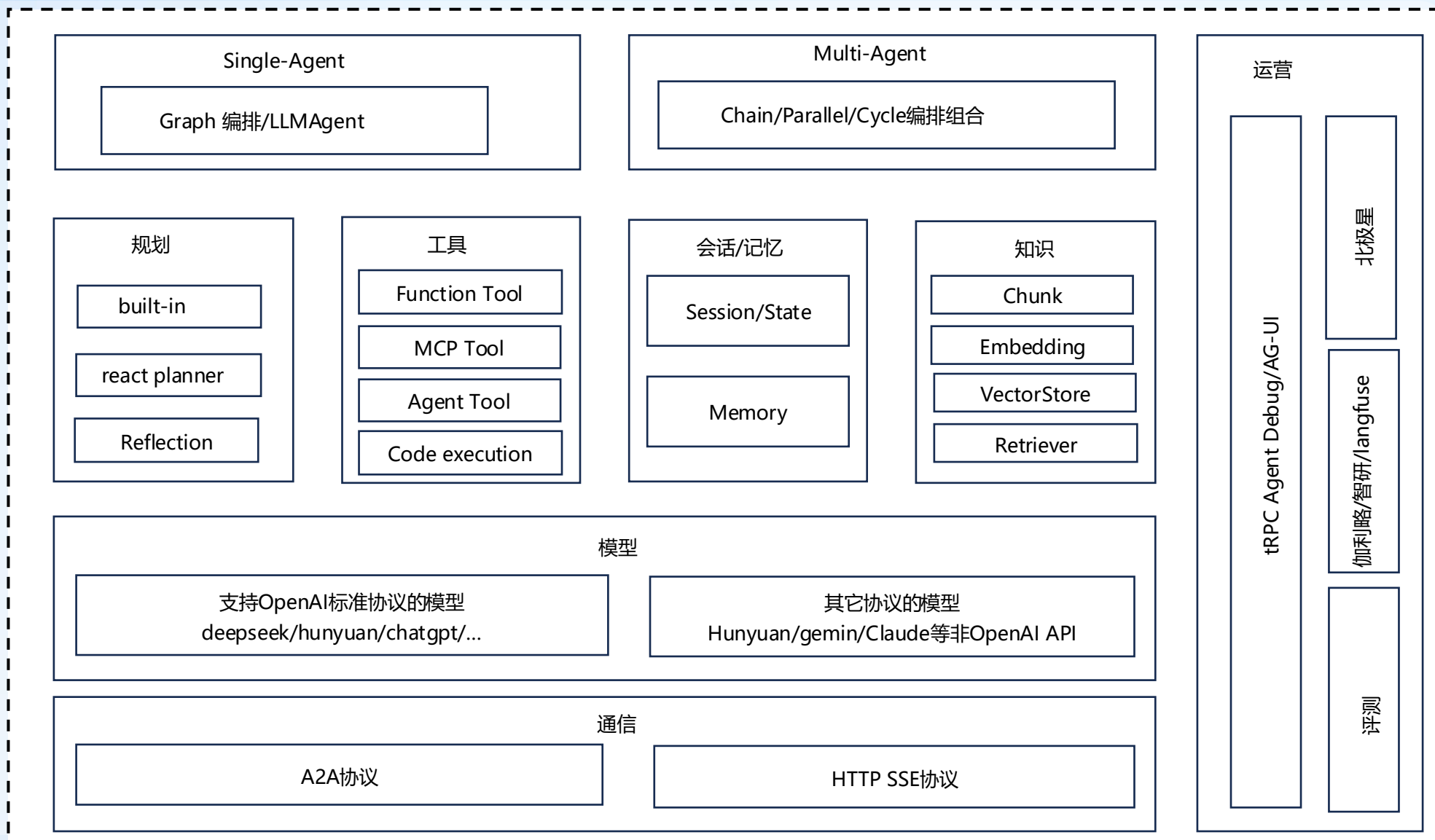


# 1. tRPC-Agent 简介

## 整体架构图

### 架构要点:

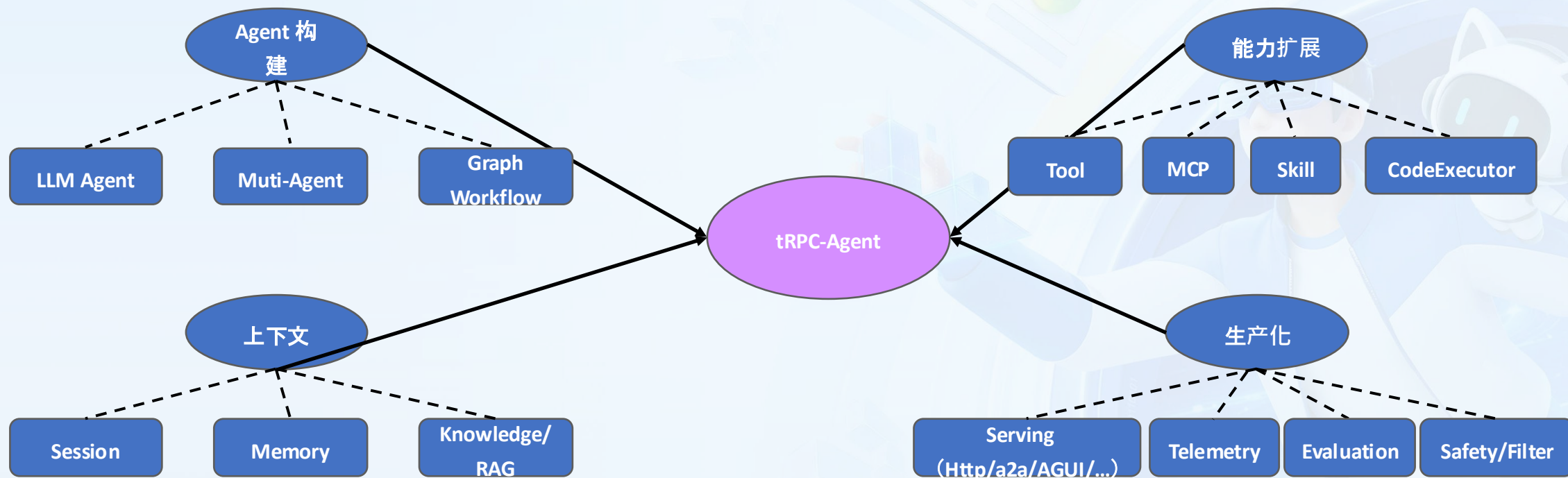
- ✓ 支持单Agent和多Agent协作, 支持图编排
- ✓ 所有组件模块均可扩展
- ✓ 和tRPC生态结合, 提供tRPC配套运营体系



# 1. tRPC-Agent 简介

## 核心定位

少量代码，组合出可上线的智能体应用。



# 1. tRPC-Agent 简介

## Demo到生产级

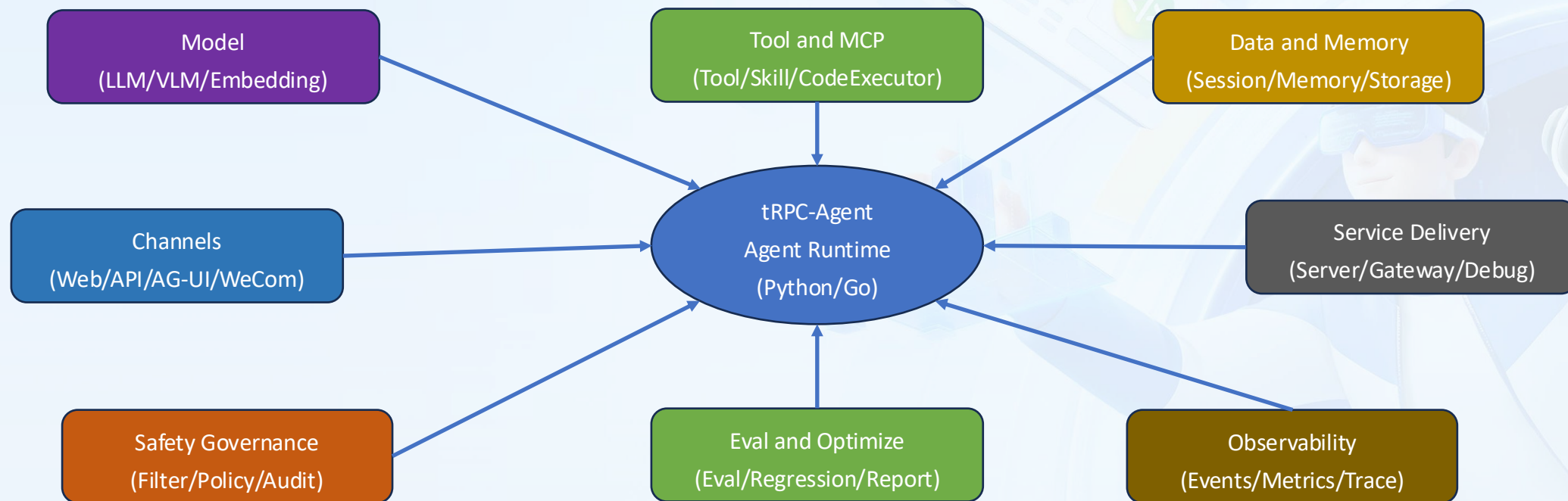
把模型、工具、记忆、流程、服务和观测接到同一条工程链路上



# 1. tRPC-Agent 简介

## 框架生态

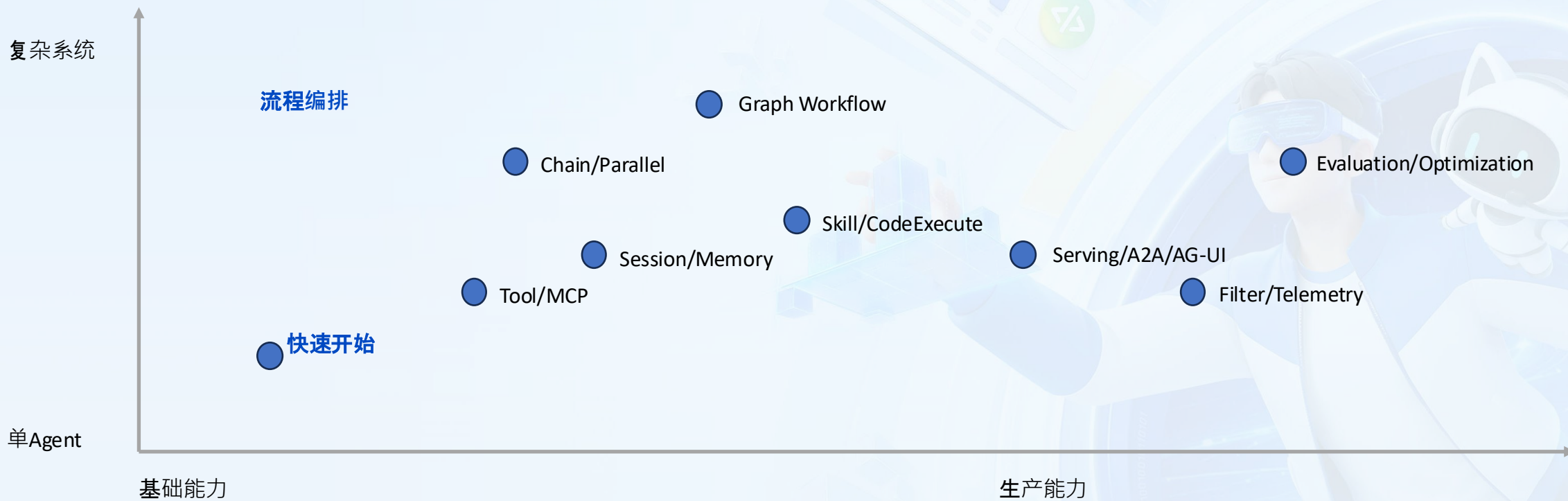
生态不是单点能力堆叠，而是模型、工具、数据、评测、治理和交付的持续协同



# 1. tRPC-Agent 简介

## 能力矩阵

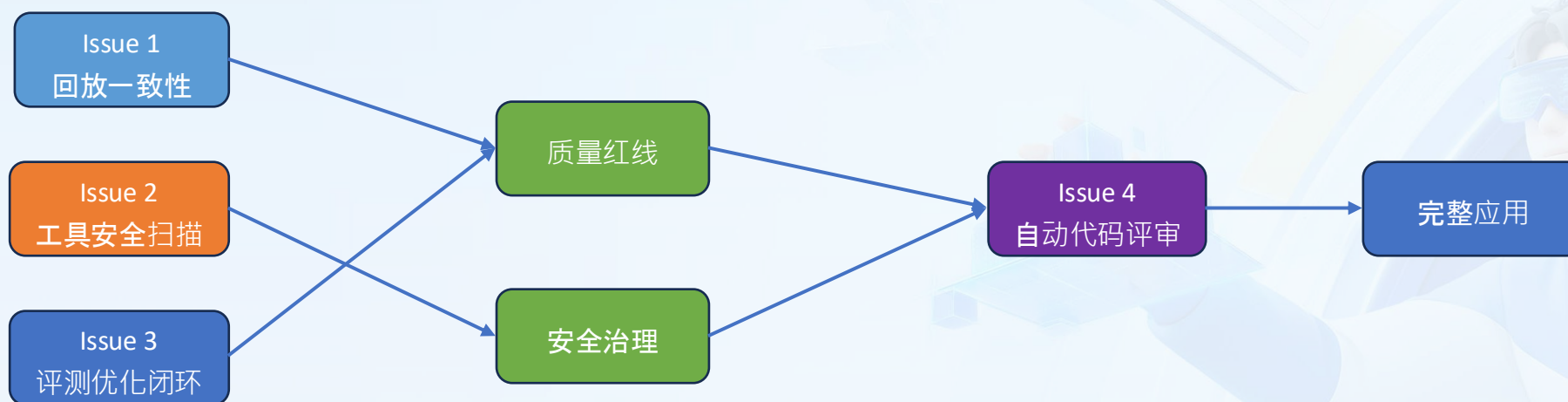
从“能跑”到“能上线”，能力逐层增强。



## 2. 上手issue

### 共建 Issues 总览

四类题目覆盖 Agent 工程最关键的三条线：状态可靠、安全边界、质量闭环

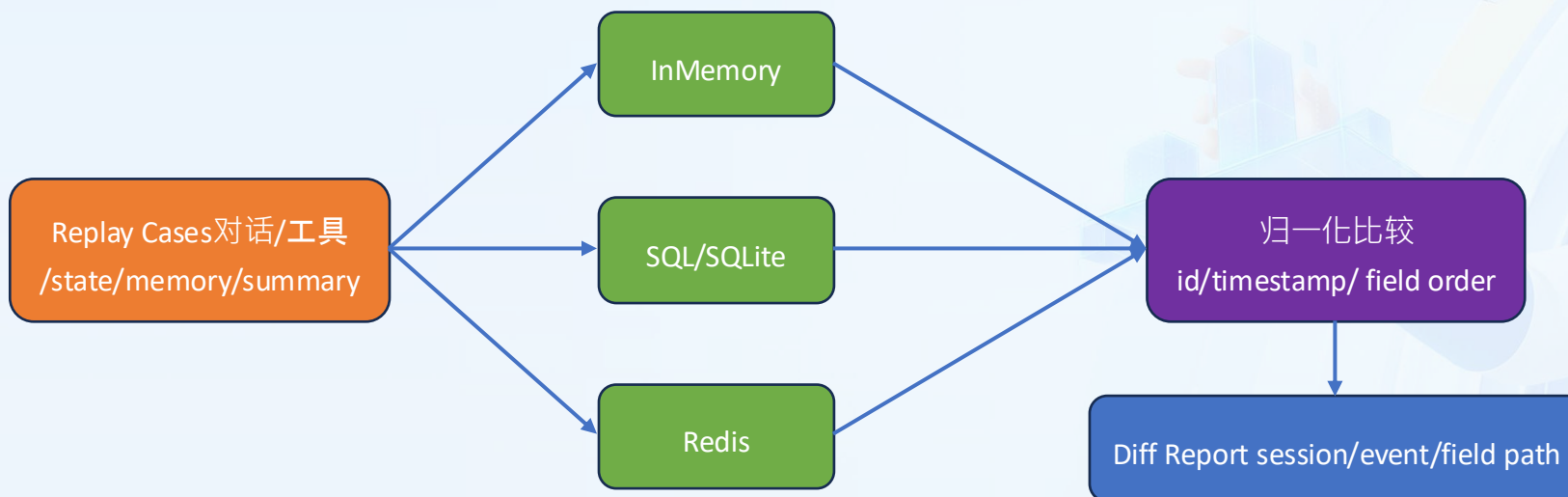


## 2. 上手issue

### Issue 1: Session / Memory 回放一致性

目标：发现后端切换后事件顺序、状态、记忆、摘要不一致的问题。

交付：测试框架、replay case、差异报告、设计说明。

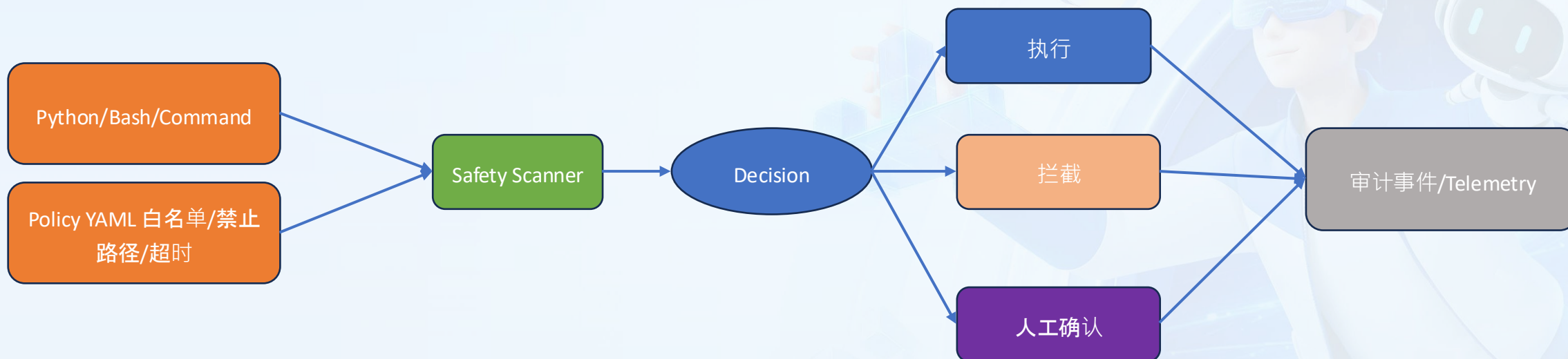


## 2. 上手issue

### Issue 2: Tool 执行安全扫描

目标：工具真正执行前，先做风险判定和可审计拦截。

重点风险：删文件、读密钥、网络外连、依赖安装、无限循环、敏感信息泄漏。

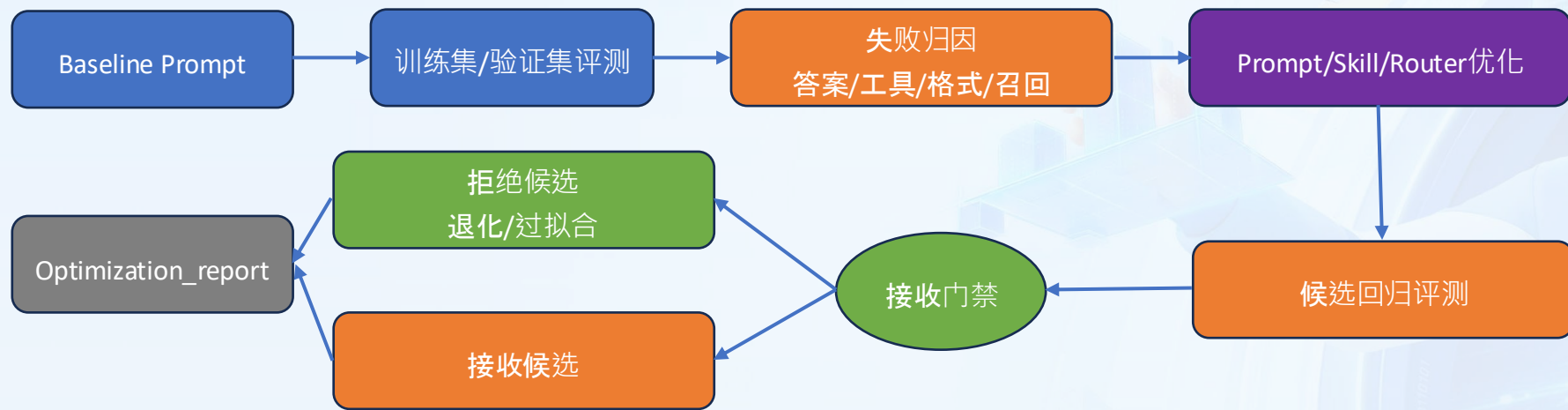


## 2. 上手issue

### Issue 3: Evaluation + Optimization 闭环

目标：让 prompt 优化变成可复现、可回归、可审计的工程流程。

验收重点：验证集不能退化，隐藏样本决策准确，报告可机器检查。

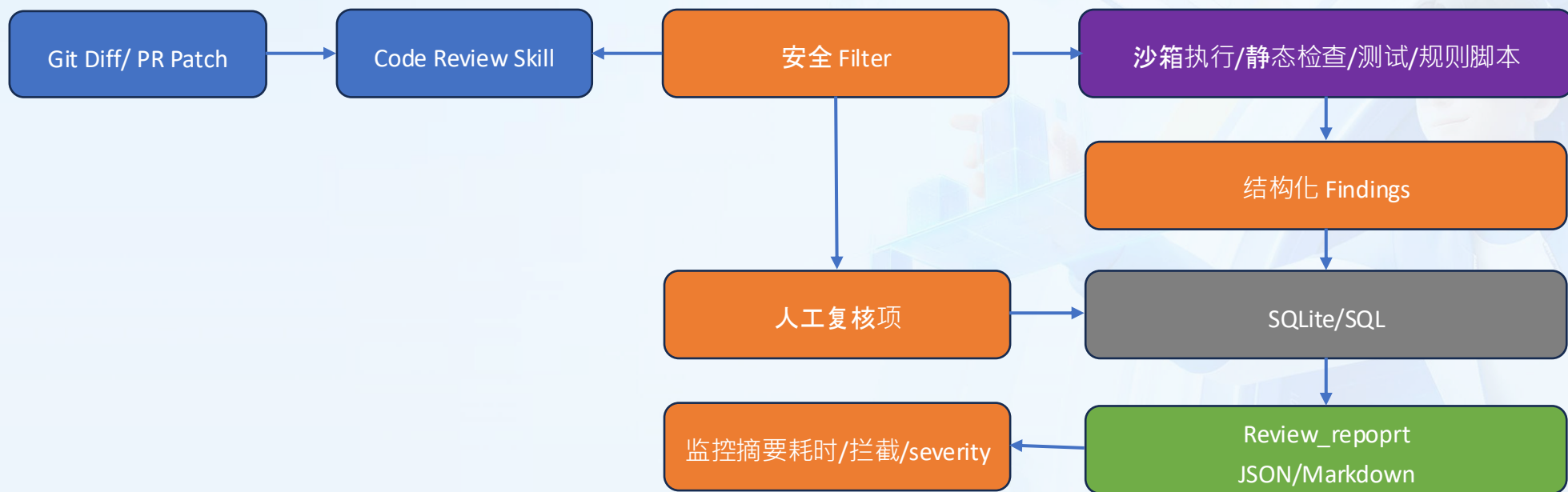


## 2. 上手issue

### Issue 4: 自动代码评审 Agent

目标：不是“让模型点评代码”，而是构建可验证的 CR Agent 系统。

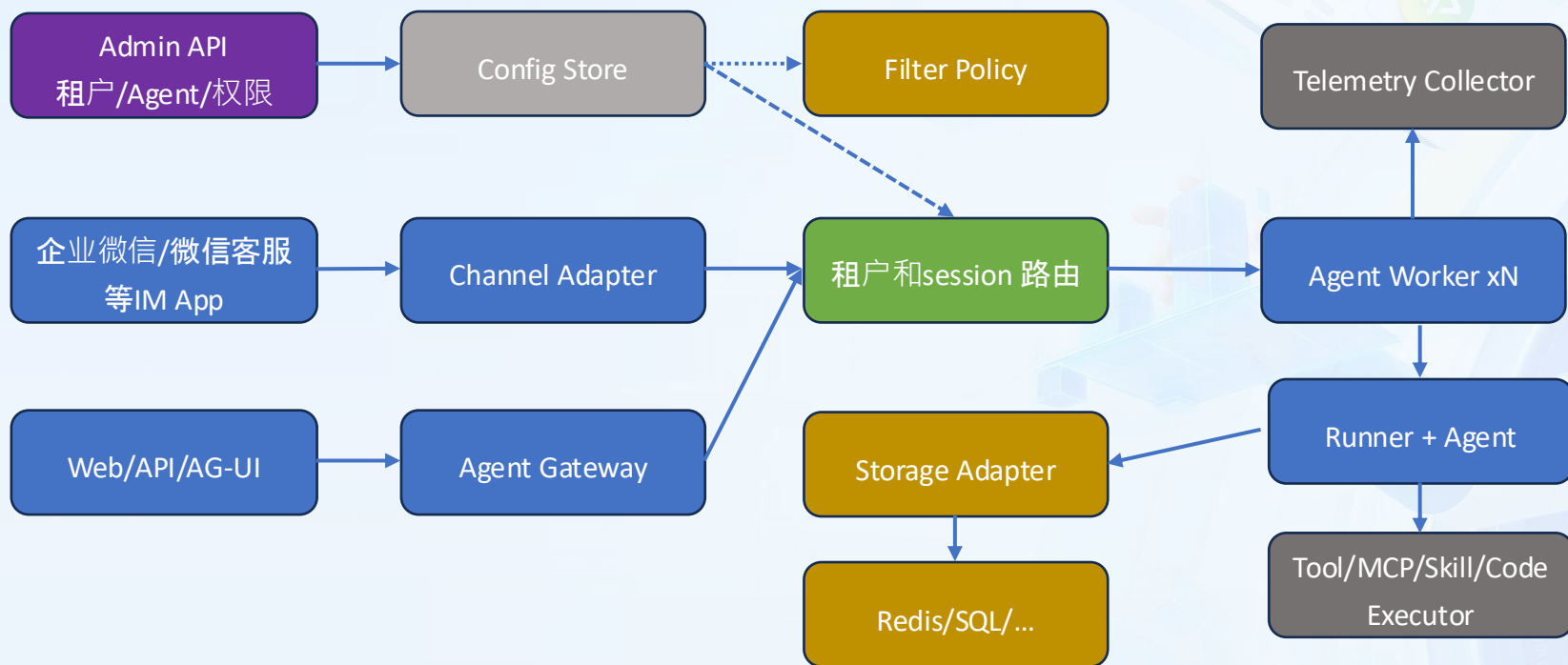
交付：Skill、沙箱脚本、存储 schema、测试样例、结构化报告。



### 3. 实战任务

## 多租户多节点化 Agent 平台

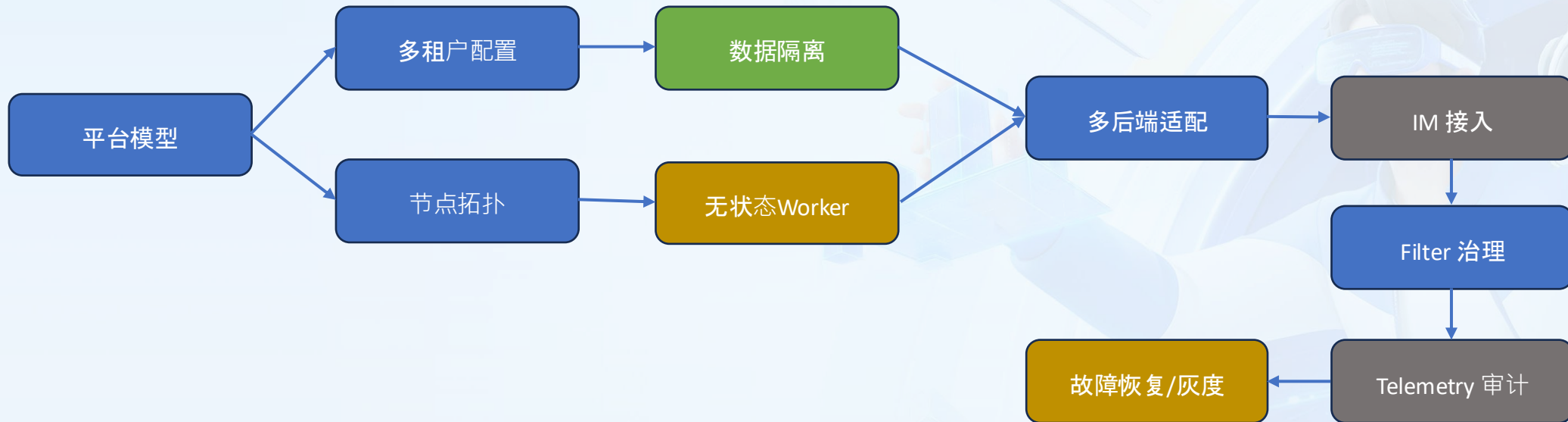
把单个 Agent 扩展成多租户、多入口、多节点、可治理的平台。



### 3. 实战任务

## 多租户多节点化 Agent 平台

任务拆解建议

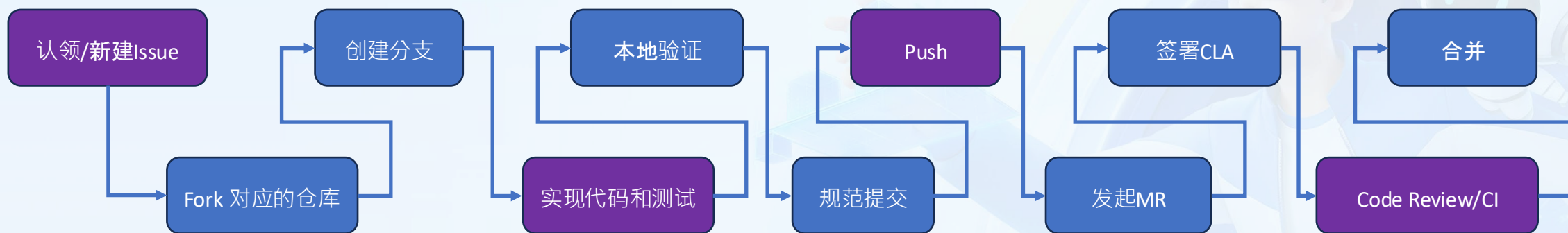


## 4. 代码提交流程

### 如何提交代码

选择仓库：

- Python 相关: <https://github.com/trpc-group/trpc-agent-python>
- Go 相关: <https://github.com/trpc-group/trpc-agent-go>



## 4. 代码提交流程

### 提交前检查清单

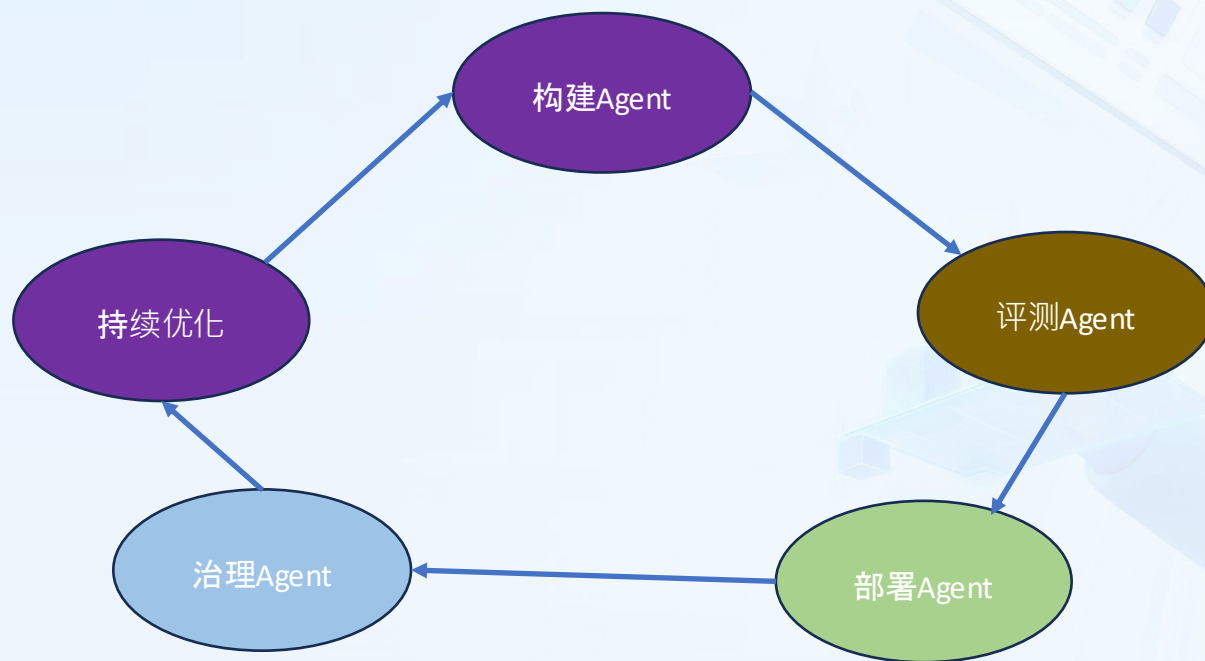
PR 描述建议包含：问题背景、方案摘要、测试结果、风险和限制、关联 Issue、Release Notes。



## 5. 总结

### 持续演进闭环

tRPC-Agent 的目标：让 Agent 应用从一次性 Demo，走向可持续演进的生产系统。



# THANKS

